

Seminar Presentation Script: Observer Pattern

Group 52

Nguyen Dinh Minh Huy (Student ID: 25125083)

July 10, 2026

Presenters

- **Speaker A:** Member A
- **Speaker B:** Member B

Presentation Flow & Dialogue

Slide 1: Title Slide (The Observer Design Pattern)

Speaker A: Hello everyone, and welcome to Group 52's presentation. Today, my colleague and I will be discussing the Observer Design Pattern—a fundamental behavioral pattern that plays a critical role in decoupling state changes and notifications in modern software. Let's start with a real-world scenario.

Slide 2: Real-world Problem (YouTube Subscription)

Speaker B: Imagine you are building a video streaming platform like YouTube. Creators upload videos to their channels, and users subscribe to these channels. When a new video goes live, subscribers need to be notified immediately. However, the challenge is that subscribers use different channels of notifications—some want an email alert, others want a mobile push notification, and some want updates on their Smart TV. Moreover, users subscribe and unsubscribe dynamically at runtime. How do we build this without tangling our codebase?

Slide 3: Naive Solution (Without Pattern)

Speaker A: In a naive solution, we might create concrete classes for each subscriber type, such as `EmailSubscriber` and `MobileSubscriber`. Our `NaiveYouTubeChannel` class then maintains separate lists for each of these concrete classes. As you can see in this code snippet, when a video is uploaded, the channel iterates through the lists of email subscribers and mobile subscribers individually, calling their specific methods like `receiveEmailNotification` or `receivePushNotification`. While this works for simple scenarios, it is highly problematic.

Slide 4: Drawbacks of the Naive Solution

Speaker B: This naive approach has three major flaws:

First, **Tight Coupling**. The channel must know the exact concrete types of subscribers and their specific update methods.

Second, it violates the **Open-Closed Principle (OCP)**. If we want to add a third subscriber type—say, `TVSubscriber`—we must modify the channel class, add a new vector list, and modify the notification loop inside `uploadVideo`.

Lastly, it is **rigid and hard to scale** as the number of subscriber channels grows.

Slide 5: Introduction to the Observer Pattern

Speaker A: To solve this, we use the **Observer Pattern**. The intent of the pattern is to define a one-to-many dependency between objects. When the Subject—in our case, the YouTube Channel—changes state, all its dependents—our Subscribers—are notified and updated automatically. The key is that the Subject only interacts with an abstract interface, not concrete classes. This cleanly separates the publisher from its subscribers.

Slide 6: General Class Diagram

Speaker B: Here is the general UML diagram of the pattern. We have:

1. The **Subject** interface, which defines methods to attach, detach, and notify observers.
2. The **Observer** interface, which defines a simple `update()` method.
3. The **ConcreteSubject**, which maintains the list of Observers and broadcasts notifications.
4. **ConcreteObservers**, which implement the `update()` method to perform their custom action.

Slide 7: Specific Class Diagram (YouTube Example)

Speaker A: Applying this to our YouTube problem, we introduce the abstract **Subscriber** interface as our Observer, which declares a single `update(channelName, videoTitle)` method. Our `YouTubeChannel` class implements the Subject interface, maintaining a single list of `Subscriber*` pointers. Now, `YouTubeChannel` has no idea whether it's notifying an email subscriber, a mobile app, or a smart TV. It simply calls `update()` on all registered pointers.

Slide 8: Code Implementation & Demo

Speaker B: Let's look at the refactored code. The `YouTubeChannel` class is now clean. To subscribe, we simply push the `Subscriber*` to the vector; to unsubscribe, we erase it. Adding a new `TVSubscriber` class requires zero changes to the channel class itself! In our console demo, you can see that when a video is uploaded, all three subscribers receive the updates. When we unsubscribe the mobile app subscriber, the subsequent upload only triggers updates for the email and TV subscribers.

Slide 9: Pros & Cons

Speaker A: Like all patterns, the Observer pattern has trade-offs. The main **Pros** are that it satisfies the Open-Closed Principle and allows dynamic runtime subscriptions. The **Cons** include the **Lapsed Listener Problem**—if observers are destroyed without unsubscribing, the subject will hold dead pointers, causing memory leaks or crashes. Additionally, developers must be careful about cascading update loops if observers trigger changes back in the subject.

Slide 10: Other Real-world Applications

Speaker B: In modern software development, you see this pattern everywhere. In web dev, browser DOM events like `addEventListener` are classic Observers. Similarly, state management libraries like Redux or Vuex notify UI components to re-render when the state store updates. In mobile dev, reactive frameworks like RxSwift, Android LiveData, and SwiftUI's `@Published` variables use Observers to keep UI views synchronized with underlying data models.

Slide 11: Interactive Quiz

Speaker A: Now, to wrap things up and keep you all awake, we have a short quiz! Correct answers will get small sweet treats, so raise your hand fast!

Speaker B: [Reads Q1] “What type of design pattern is the Observer Pattern?” ... [Wait for hands] ... Correct! It is Behavioral. Here is your candy!

Speaker A: [Reads Q2] “Which SOLID principle is satisfied when adding new subscribers without changing the channel code?” ... Yes! Open-Closed Principle. Excellent.

Speaker B: [Reads Q3] “What problem occurs when a listener is deleted without unsubscribing?” ... Correct, the Lapsed Listener problem leading to memory leaks.

Slide 12: Conclusion & References

Speaker A: In conclusion, the Observer pattern is a powerful tool to build scalable, loosely-coupled event-driven systems. Thank you all for listening. We are now open to any questions or reviews from the audience.